

[15] Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers

저자: Kunal Sawarkar, Abhilasha Mangal, Shivam Raj Solanki 학회/년도: IEEE MIPR (Multimedia Information Processing and Retrieval) 2024 분량: 약 10페이지 역할군: (A) VAQ 동기 부여

요약

Blended RAG는 RAG(Retrieval-Augmented Generation) 시스템의 검색 정확도를 높이기 위한 하이브리드 검색 전략을 제안한다. 핵심 발견은 벡터 검색만으로는 부족하며, 키워드 검색·희소 인코딩·밀집 벡터 검색을 결합해야 한다는 것이다.

이 논문이 보여주는 중요한 시사: 1. VAQ의 복잡성 증가: 벡터 검색이 단순 top-k에서 벗어나 복합 검색 전략으로 진화 중 2. 적응적 카디널리티 추정의 필요성: 각 검색 전략의 결과 크기가 쿼리마다 크게 달라지므로, 고정 선택도는 특히 치명적 3. 실무 요구사항의 복잡성: 단순 학술 벤치마크와 달리 실제 RAG 시스템은 여러 검색 전략을 결합 Blended RAG의 성능 향상은 modest하지만(5~8%), 검색 결과를 관계형 데이터와 조인하고 필터링하면 VAQ가 되며, 이때 정확한 카디널리티 추정의 가치가 극대화된다.

상세분석

15.1 해결하고자 하는 문제: RAG 시스템의 검색 품질

RAG(Retrieval-Augmented Generation)란?

RAG는 다음 절차로 동작한다:

사용자 질문 → 검색 (관련 문서 찾기) → LLM 입력 (검색 결과 + 질문) → 생성된 답변

검색 단계가 LLM 응답 품질의 상한선(ceiling)을 결정한다: - 검색 결과가 정확하면 → LLM이 좋은 답변 생성 가능 - 검색 결과가 부정확하거나 불완전하면 → LLM이 아무리 좋아도 제한된 답변만 가능

단일 검색 전략의 한계:

기존 RAG 시스템은 한 가지 검색 방식만 사용했다:

- 1. 키워드 검색(BM25)만 사용:** - 장점: 정확한 단어 매칭에 강함 (예: "Apple Inc." 검색 시 "Apple" 회사만 찾음) - 단점: 의미적 유사성을 못 잡음 - 질문: "자동차 사고" - 찾지 못하는 문서: "교통 충돌", "차량 사건" (의미가 같지만 단어가 다름)
- 2. 밀집 벡터 검색(KNN)만 사용:** - 장점: 의미적 유사성을 잘 포착 (BERT, GPT 임베딩 모델 활용) - 단점: 정확한 키워드 매칭에 약함 - 질문: "Python 3.11 버전의 async 함수" - 문제: "Python"과 "3.11"이라는 정확한 숫자를 벡터로 정확히 표현하기 어려움 - 의미적으로는 관련 있지만 **잘못된 버전**(예: Python 2.7)의 문서를 반환할 수 있음
- 3. 희소 인코딩(ELSER)만 사용:** - Elastic이 개발한 학습된 희소 인코더 - 장점: BM25보다 유연하고 의미도 어느 정도 포착 - 단점: 복잡한 의미 관계를 완전히 포착하지 못함

15.2 핵심 아이디어: 세 가지 검색 전략의 혼합

Blended RAG는 세 가지 검색 기능을 동시에 실행하고 결과를 병합한다:

1. BM25 (키워드 기반):

TF-IDF 확장 알고리즘

$$\text{점수} = (\text{tf_in_doc} * \text{IDF}) / (\text{tf_in_doc} + k1 * (1 - b + b * \text{doc_len} / \text{avg_len}))$$

- tf_in_doc: 문서에서 단어 출현 빈도
- IDF: 역 문서 빈도 (흔한 단어일수록 낮음)
- k1, b: 튜닝 파라미터

2. KNN 밀집 벡터 (시맨틱):

```
embedding = text_encoder(query)
top_k = argmax_k(cosine_similarity(embedding, doc_embeddings))
```

- BERT, GPT-3 같은 사전 학습 모델로 텍스트를 벡터로 변환
- 코사인 유사도로 상위 k개 문서 반환

3. ELSER 희소 인코더 (하이브리드):

```
sparse_vector = elser_model(query)
점수 = inner_product(sparse_vector, doc_sparse_vector)
```

- 키워드와 시맨틱의 중간 지점
- 학습된 가중치로 중요한 단어에 높은 점수 부여

15.3 세 결과의 병합: RRF (Reciprocal Rank Fusion)

세 검색기로부터 받은 결과를 **Reciprocal Rank Fusion (RRF)**으로 병합한다:

$$RRF_score = \sum (1 / (rank + k))$$

예시: - BM25 결과: [문서A (rank 1), 문서B (rank 2), 문서C (rank 5)] - KNN 결과: [문서B (rank 1), 문서A (rank 3), 문서D (rank 2)] - ELSER 결과: [문서A (rank 1), 문서D (rank 2), 문서B (rank 4)]

문서A의 RRF 점수 = $1/(1+60) + 1/(3+60) + 1/(1+60)$ = 높음 → 최종 순위 1위

이 방식의 장점: - 한 검색기가 부정확해도 다른 검색기가 보완 - 여러 검색기에서 모두 높은 순위를 받은 문서가 최종적으로 우위를 가짐

15.4 하이브리드 쿼리 변형

문제 유형에 따라 다양한 병합 전략을 사용할 수 있다:

전략	사용 시점	예시
best-fields	한 필드에서 가장 좋은 매칭이 중요할 때	문서 제목에 모든 키워드가 있는 경우 우위
cross-fields	여러 필드에 걸쳐 키워드 분산 시	"문제"는 제목에, "해결책"은 본문에 있는 경우
most-fields	여러 검색기에서 매칭되는 것을 우위로	BM25, KNN, ELSER 모두에서 매칭되는 문서
phrase-prefix	구문 매칭과 접두사 검색 결합	"machine learning" 구문 + "algor*" 매칭

15.5 성능 결과: 모델별 향상

NQ 데이터셋 (자연어 질문): - 기존 최고 (BM25 또는 단일 KNN): NDCG@10 = 0.633 - Blended RAG: NDCG@10 = 0.67 - 향상률: **+5.8%**

TREC-COVID 데이터셋 (의료 정보): - 기존 최고: NDCG@10 = 0.804 - Blended RAG: NDCG@10 = 0.87 - 향상률: **+8.2%**

향상 폭이 크지 않아 보이지만, 이는 기존 최고 성능이 이미 높기 때문이다. 절대 성능 관점에서는 상당한 개선이다.

더 중요한 것은: - **재현율(Recall)**도 함께 향상됨: 관련 문서를 놓치는 경우가 줄어들음 - **견고성(Robustness):** 쿼리 타입이 변해도 안정적인 성능

15.6 본 논문과의 관계: VAQ 복잡성의 증가

Blended RAG가 보여주는 것:

1. 벡터 검색이 단순 top-k를 넘어 복잡해지고 있다:

```
-- 기존 단순 벡터 검색
SELECT * FROM documents
WHERE embedding <=> query_vector
LIMIT 10;

-- Blended RAG 스타일의 복합 검색
SELECT *,
      (bm25_score + knn_score + elser_score) / 3 AS blended_score
FROM (
  SELECT * FROM bm25_search(query) UNION ALL
  SELECT * FROM knn_search(query) UNION ALL
  SELECT * FROM elser_search(query)
) unified_results
GROUP BY doc_id
ORDER BY blended_score DESC
LIMIT 10;
```

이것을 관계형 데이터와 조인하면:

```
-- 실제 필요한 쿼리 (VAQ의 예시)
SELECT documents.*, users.rating, categories.category_name
FROM documents
JOIN users ON documents.author_id = users.id
JOIN categories ON documents.category_id = categories.id
WHERE (
  -- 벡터 검색
  bm25_score(documents.text, query) > 0.5
  OR knn_similarity(documents.embedding, query) > 0.8
) AND users.rating >= 4.0
ORDER BY combined_relevance DESC
LIMIT 10;
```

이 쿼리에서: - 세 개의 검색 함수(bm25_score, knn_similarity, elser_score)의 결과 크기가 불확실 - 각 검색 함수의 선택도에 따라 JOIN의 효율성이 크게 달라짐 - **고정 선택도 추정은 특히 치명적**

2. 각 검색 전략의 결과 크기가 쿼리마다 크게 달라진다:

쿼리 1: "Python 프로그래밍" → BM25는 10,000개, KNN은 50개 (BM25 편향) 쿼리 2: "기계학습 개요" → BM25는 1,000개, KNN은 5,000개 (KNN 편향) 쿼리 3: "네트워크 보안" → 세 검색기 모두 비슷한 크기

고정 33.3% 선택도로는 이런 변화를 절대 포착할 수 없다.

3. 검색 결과의 품질이 후속 조인에 영향:

Blended RAG의 결과가 높은 품질이라면, 이를 관계형 데이터와 조인할 때: - 검색 결과가 많을 때: Hash Join이 효율적 - 검색 결과가 적을 때: Nested Loop Join이 효율적

정확한 카디널리티 추정이 없으면, 옵티마이저가 최악의 JOIN 방식을 선택할 수 있다.

15.7 Blended RAG의 한계와 Exqutor의 가치

Blended RAG의 한계:

1. RRF 가중치(k 값)를 데이터셋별로 수동 튜닝해야 함
2. 세 검색 전략 모두를 항상 실행하므로 **3배의 검색 오버헤드** (병렬 처리로 일부 상쇄)
3. 검색 결과의 품질 편차가 큼 (쿼리마다 결과 크기 큰 변동)

Exqutor의 적용 시나리오:

Blended RAG + 관계형 데이터 조인 + **Exqutor의 ECQO**:

1. Blended RAG 실행 → 검색 결과 (크기 불확실)
2. ECQO가 각 검색 전략의 실제 카디널리티 측정
3. 정확한 정보로 JOIN 계획 수립
4. 최적의 JOIN 방식 선택 → 극적인 성능 향상

추가 제기 문제

1. **RRF 가중치 튜닝**: Blended RAG의 성능은 RRF 합병 파라미터에 크게 의존한다. 데이터셋별, 도메인별로 최적 가중치가 다르므로, 자동으로 가중치를 학습하는 방법이 필요하다.
2. **검색 지연 시간**: 세 검색 전략을 모두 실행하면 지연 시간이 3배 증가한다. 병렬 처리로 일부 상쇄되지만, 여전히 개선의 여지가 있다.
3. **메모리와 저장 공간**: 세 가지 인덱스(BM25, KNN 벡터, ELSER 희소)를 모두 유지해야 하므로, 저장 공간이 상당히 증가한다.
4. **향후 확장성**: Blended RAG가 더 많은 검색 전략(예: 시맨틱 그래프 검색, 속성 필터링 기반 검색)을 추가하면, 결과 병합의 복잡도가 기하급수적으로 증가한다. 이때 정확한 카디널리티 추정은 더욱 중요해진다.